

PROJECT OVERVIEW

For our applied machine learning exam, we have chosen project 3: *Plant Disease Recognition from Leaf Images*. The dataset consists of 87976 images of healthy and diseased leaves which are categorized into 38 distinct classes. Our objective is to evaluate the performance of various architectures to determine their efficiency and accuracy in complex classification tasks.

EXPLORATORY DATA ANALYSIS

The split consists of 70333 training images, 17610 validation images and a mere 33 test images. The images are all 256x256 resolution. The leaves are, most commonly, in the centre of the image whilst their backgrounds are varying. The images are in the RGB colour space. There's an average of 120.34 of red channels, 110.91 blue channels and 147.76 green channels. The green channels being the dominating colour channel is consistent with the botanical nature of the dataset.



Figure 1: Sample from test data



Figure 2: Sample image from validation data



Figure 3: Sample image from test data

By using the Laplacian variance, we were able to explore the image quality. The threshold for the image to be considered “blurry” was set to 100 – as a common starting point. 419 images from the training set were below the threshold – making up 0.60% of the dataset. For the validation dataset, 104 images were below the threshold, also making up 0.60%. For the test dataset, only 2 images were below the threshold but due to the size of the dataset, it makes up for 6%.

After some consideration, we've chosen a *data augmentation by inclusion*-approach by leaving all images in our dataset – despite the slight variation in the image quality. This was done to ensure architectural robustness and mimic the variation there might be when working with real life data.

METHODOLOGY AND MODEL ARCHITECTURE

The model architectures used to classify the leaves are a custom CNN, ResNet, MobileNet, ViT and a convolutional autoencoder using both PyTorch and TensorFlow as our framework. The parameters described are those that yielded the best performance.

CNN

The convolutional block consists of 3 in channels, representing the colour channels of the images, with 32 out channels, reflecting the number of feature maps desired. The kernel size is set to 3. Padding is set to 1 to prevent edge-pixel loss. The convolutional block is followed by Batch Normalization, to stabilize training; ReLU as activation function and ended off with a Max Pooling of 2 to extract the pixels of strongest value.

The next convolutional block has 32 in channels, 64 out channels. Kernel size and padding are unchanged. The stride has been introduced and set to 1 to balance data reduction and detail preservation. The block is followed by batch normalization, ReLU and a MaxPool.

The final convolutional layer has 64 in channels, 128 out channels. Kernel size and padding are unchanged. The stride has been increased to 2 to reduce spatial dimension while allowing the model to learn its own down-sampling parameters. The block is followed by BatchNorm, ReLU and a MaxPool.

The convolutional blocks are then followed by Adaptive Max Pooling to ensure the output matches the size required for the next step – the fully connected layer.

The images 128 feature maps from the convolutional layers are flattened into a single vector and fed into a hidden layer of size 256. The layer is followed by a ReLU, BatchNorm and a DropOut of 0.5. The 256 layers are then what is used to determine the class of the image.

Transfer learning

The models used are ResNet18 and MobileNetV3Large. We used the transformers library to set training arguments. The evolution strategy was based on the number of epochs, learning rate to 1e-3, batch size to 32, weight decay to 1e-2. We used accuracy as the metric for the best model. We set the early stop patience to 3. The number of epochs were set to a mere 5 since it is only the classifier, we are training. The training of the classifier on ResNet18 resulted in an accuracy of 93.11% with a loss of 0.2974. For MobileNetV3Large, the accuracy was 98.15% with a loss of 0.0059.

A similar approach was used for fine tuning. We froze the last three layers and lowered the learning rate to $1e-5$. The fine-tuning of ResNet18 resulted in an accuracy of 94.70% with a loss of 0.2038. For MobileNetV3Large we got an impressive 99.24%% with a 0.0238 loss

Autoencoder

We have built an Autoencoder and from there, extracted the latent features.

For the Random Forest model, we have set number of estimators to 300; criterion was based on Gini-Index, with a maximum depth of 15 and random state was set to 42. This resulted in an accuracy of 73.35%. The parameters set for Logistic regression were a random state of 42, $C = 0.01$, the solver was set to saga, penalty was ElasticNet with 1000 max iterations. This resulted in an accuracy of 68.32%. Lastly, we built a 4-layer Feed forward neural network with batch normalization and drop out as regularization methods. The FFNN had an accuracy of 25.59%.

ViT

Using the OpenAI ViT we used the same approach as earlier. Due to the computational expensiveness of this task, the model continuously crashed after 3 epochs. The accuracy after training for 3 epochs was an impressive 93.58% with a loss 0.4659.

MODEL COMPARISON AND INTERPRETABILITY

The model of the best performance in the task of classification of plant diseases was the fine-tuned version of MobileNetV3Large achieving and accuracy of 99.24%. The worst performing model was the feed forward neural network built on latent features extracted from the convolutional autoencoder, with an accuracy of 25.59%.

To further our understanding in the model's predictions, we used SHAP. By using SHAP we see that things like discolorations and texture change are the main class indicators. This is illustrated in the heat map by the clusters of red pixelations. The blue pixels indicate the contrary; these symptoms may be associated with another disease. The remaining opaque area holds little to no value in the model's final prediction.

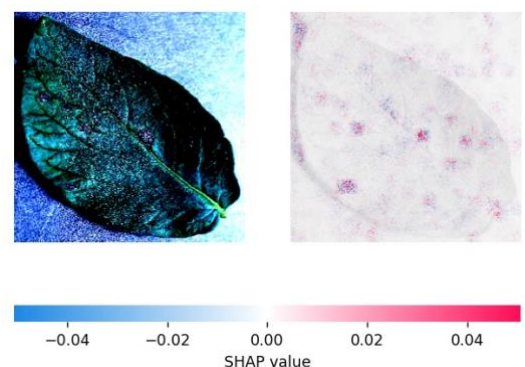


Figure 1: SHAP interpretation